

Lucrarea nr. 6 : *Studiul registrelor de deplasare*

1. Scopul lucrării

În partea teoretică se face o prezentare succintă a structurii interne a principalelor tipuri de registre, după care sunt prezentate câteva registre realizate în structuri integrate (se pune accent pe semnificația pinilor și pe unele particularități ce apar în funcționarea acestor circuite).

Ca aplicații ale registrelor de deplasare se studiază: realizarea divizoarelor de frecvență; realizarea luminilor dinamice și a generatoarelor de numere pseudo-aleatoare. O parte din aceste aplicații sunt prezentate ca implementări cu structuri integrate distincte sau ca descrieri în limbaj VHDL.

2. Considerente teoretice

2.1. Introducere

La nivel de bit, memorarea informației se face cu ajutorul latch-urilor sau a bistabililor. La nivel de cuvânt, memorarea și procesarea informației se face cu ajutorul registrelor. Registrul este un circuit logic secvențial format dintr-o succesiune de n latch-uri sau bistabili.

Încărcarea unui registru (operația de introducere a cuvântului de n biți în registru) se poate face în două moduri:

- *serial* – încărcarea se face bit cu bit în ritmul unui semnal de ceas, motiv pentru care timpul de încărcare este egal cu n perioade de ceas;
- *paralel* – toți biții sunt introduși în același timp, pe tranziția activă a unui semnal de ceas.

Extragerea informației din registru (operație denumită citire) se poate face tot în două moduri:

- *serial* – citirea se face bit cu bit pe n tranziții active ale unui semnal de ceas;
- *paralel* – toți biții sunt citiți în același timp.

În general, un registru este definit ca o structură liniară de celule de memorare ce prezintă una sau mai multe din următoarele funcții: - acces serial; - acces paralel; - ieșire serială; - ieșire paralelă.

Clasificarea registrelor:

Clasificarea registrelor se poate face după mai multe criterii însă cel mai important este cel care indică modul de intrare și de ieșire a informației.

- După tipul celulelor de memorie utilizate:
 - cu latch-uri;
 - cu bistabili;
- După tipul celulelor de memorie utilizate:
 - cu intrare paralelă și ieșire paralelă (parallel in/parallel out);
 - cu intrare serială și ieșire paralelă (serial in/parallel out);
 - cu intrare paralelă și ieșire serială (parallel in/serial out);
 - cu intrare serială și ieșire serială (serial in/serial out);

Registrele cu încărcare serială mai sunt denumite și *registre de deplasare* și, la rândul lor, pot fi împărțite în două categorii: a) registre cu deplasare la dreapta și b) registre bidirecționale.

Registrele prevăzute cu ambele tipuri de intrări (serie și paralel) și ieșire paralelă sunt denumite registre universale.

2.2. Structura internă a registrelor

• **Registru cu intrare paralelă și ieșire paralelă (registru paralel/paralel).** O structură de registru cu încărcare paralelă și ieșire paralelă se obține relativ ușor prin utilizarea unui număr convenabil de bistabili de tip D. Un exemplu de registru paralel/paralel pe 4 biți se prezintă în figura 1.

Funcționare:

- toate intrările de ceas ale bistabililor se conectează împreună și formează intrarea de comandă a încărcării paralele **CKP**;
- la apariția unei tranziții active pe intrarea **CKP**, starea logică a intrărilor **PI** este copiată în celulele de memorie și va fi păstrată nealterată până la următoarea tranziție activă;
- încărcarea paralelă se poate face numai atunci când pe intrarea **CKP** se aplică tranziție activă, spunem că încărcarea paralelă se face sincron cu semnalul aplicat pe **CKP**;
- intrările de tip D joacă rol de intrări paralele de date;
- ieșirile bistabililor joacă rol de ieșiri paralele de date;
- citirea informației din registru se poate face oricând;
- intrarea de ștergere **MR** (*Master Reset*), este activă pe zero logic și este folosită pentru ștergerea registrului (aducerea în starea $Q=0$ a tuturor bistabililor din structură);
- ștergerea registrului se poate face oricând prin aducerea intrării **MR** în starea 0, spunem că ștergerea se face asincron față de semnalul aplicat pe intrarea **CKP**;

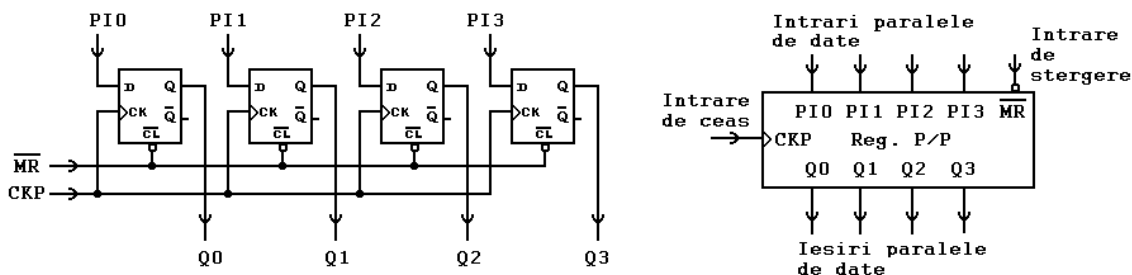


Fig. 1. Structura internă și simbolul pentru un registru paralel/paralel pe 4 biți

• **Registru cu intrare serie și ieșire paralelă (registru serie/paralel).**

O structură posibilă de registru cu încărcare serială se prezintă în figura 2. Analizând schema de conectare se constată că acest tip de registru prezintă ambele tipuri de ieșiri, atât paralelă cât și serială.

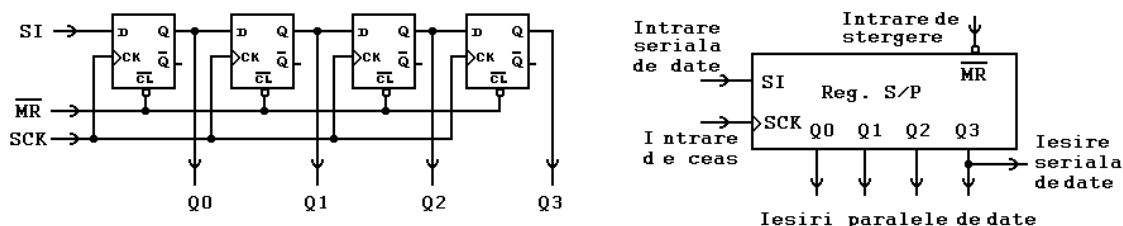


Fig. 2. Structura internă și simbolul pentru un registru serie/paralel pe 4 biți

Funcționare:

- intrarea D a primului bistabil joacă rol de intrare serială de date, **SI** (Serial Input);
- toate intrările de ceas ale bistabililor se conectează împreună și formează intrarea de comandă a încărcării seriale **SCK** (Serial Clock);
- la apariția unei tranziții active pe intrarea **SCK**, starea logică a intrării **SI** este copiată în primul bistabil iar conținutul celulelor de memorie este deplasat spre dreapta cu o poziție (de aici și denumirea de registru de deplasare);
- încărcarea acestui registru nu se poate face decât în mod serial și necesită 4 perioade de ceas (în fig. 3 se prezintă modul de încărcare serială a informației 1111 într-un registru ce prezenta inițial informația 0000, pentru a putea urmări deplasarea informației în registru au fost utilizate fonturi diferite pentru fiecare bit de intrare);
- ieșirile bistabililor joacă rol de ieșiri paralele de date;
- ieșirea ultimului bistabil joacă și rol de ieșire serială;
- citirea informației din registru se poate face oricând, independent de **SCK**;
- intrarea de ștergere **MR** (Master Reset), este activă pe zero logic și are o execuție asincronă;

Registrele de deplasare sunt foarte utile în realizarea rapidă a operațiilor de înmulțire/împărțire cu puteri ale lui 2. Se poate arăta că înmulțirea unui număr binar (stocat în registru) cu numărul 2^p înseamnă deplasarea spre stânga cu p poziții iar împărțirea cu 2^p necesită deplasarea spre dreapta cu p poziții.

Exemplu:

- scriere zecimală: $5 \times 4 = 20 \Leftrightarrow 5 \times 2^2 = 20$, deci trebuie efectuată o deplasare spre stânga cu două poziții;
- scriere binară: $101 \times 100 = 10100$
- conținutul inițial al registrului: 101
- după două deplasări la stânga, cu introducerea de zerouri pe pozițiile rămase libere se obține: **10100**;

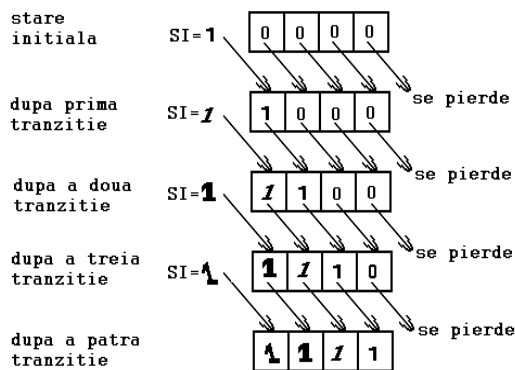


Fig. 3. Exemplu de încărcare serială a unui registru de 4 biți

• **Registru universal.**

Dacă în fața fiecărui bistabil din schema prezentată în figura 2 se introduce un multiplexor cu 2 intrări, se permite conectarea intrării D fie la intrarea de încărcare paralelă, fie la ieșirea bistabilului de pe poziția anterioară. Se obține astfel un registru universal, un registru ce se poate încărcă paralel sau serie și poate fi citit serie sau paralel.

Un exemplu de registru universal pe 4 biți se prezintă în figura 4.

Funcționare:

- modul de încărcare a registrului depinde de starea logică a intrării $S/\bar{P}$, dacă $S/\bar{P} = 0$ încărcarea se face paralel iar dacă $S/\bar{P} = 1$ încărcarea se face serie;
- multiplexoarele 2:1 se comportă ca niște comutatoare ce conectează intrările D fie la PI, fie la ieșirile bistabililor de pe poziția anterioară;
- ambele moduri de încărcare se fac sincron cu semnalul aplicat pe intrarea **CKU**;
- citirea informației din registru se poate face paralel sau serie în mod independent de **CKU**;

Facem precizarea că unele variante de registre universale au intrări separate de ceas pentru încărcarea serie respectiv paralel.

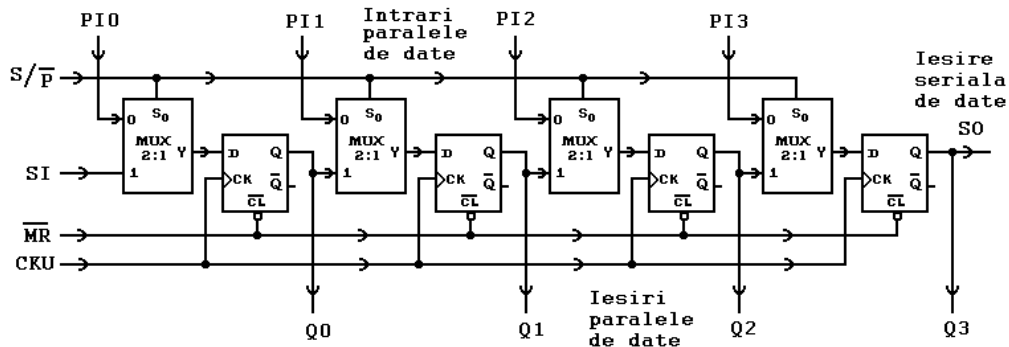


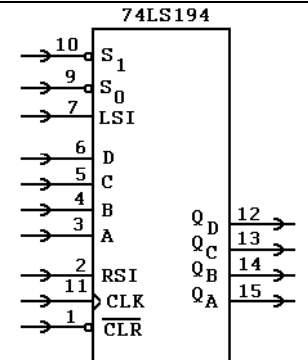
Fig. 4. Structura internă pentru un registru universal pe 4 biți

2.3. Exemple de registre realizate în structuri integrate

Observații	Simbol
Registru 74LS171 - circuitul conține 4 bistabili de tip D, conectați într-o manieră similară celei din figura 1; - intrarea de ștergere ($\overline{CLR}$), este activă pe zero logic; - ieșirile de date sunt disponibile și sub formă negată; - încărcarea paralelă se face pe tranziția pozitivă a semnalului de ceas;	
Registru 74LS377 - circuitul conține 8 bistabili de tip D; - intrarea de validare a accesului informației la celulele de memorie ($\overline{EN}$), este activă pe zero logic; - încărcarea paralelă se face pe frontul pozitiv al semnalului de ceas numai atunci când $\overline{EN} = 0$; - dacă $\overline{EN} = 1$, informația de intrare nu are acces spre celulele de memorare, indiferent de starea semnalului de ceas;	
Registru 74LS670 - circuitul dispune de 4 registre de 4 biți fiecare; - ieșirile sunt de tip tristate; - intrarea de validare a scrierii ($\overline{WE}$) și cea de validare a citirii ($\overline{RE}$) sunt active pe zero logic; - datorită faptului că avem semnale diferite de validare pentru citire și pentru scriere, este posibilă scrierea unei locații simultan cu citirea alteia; - adresarea registrului se face prin aplicarea unui cod binar pe liniile RB, RA (dacă este operație de citire) respectiv WB, WA (dacă este operație de scriere); - circuitul se comportă ca o memorie RAM cu acces dual.	
Registru 74LS95 - modul de lucru se alege prin intermediul intrării CM: - $CM = 0$, deplasare spre dreapta în ritmul tranzițiilor negative ale semnalului aplicat pe intrarea CKS; - $CM = 1$, încărcare paralelă în ritmul tranzițiilor negative ale semnalului aplicat pe intrarea CKP; - intrarea serială de date este SI; - poate fi folosit ca : - registru cu intrare serie și ieșire paralel; - registru cu intrare serie și ieșire serie; - registru cu intrare paralelă și ieșire paralelă; - registru bidirecțional (necesită conexiuni externe);	

Registru universal 74LS194

- registru universal pe 4 biți;
- modul de lucru se alege prin intermediul intrărilor de selecție $S_1 S_0$, după cum urmează:
 - $S_1 S_0 = 00$, memorare
 - $S_1 S_0 = 01$, deplasare informație de la $Q_A \rightarrow Q_D$
 - $S_1 S_0 = 10$, deplasare informație de la $Q_D \rightarrow Q_A$
 - $S_1 S_0 = 11$, încărcare paralelă ($Q_D Q_C Q_B Q_A = DCBA$)
- intrarea de ștergere asincronă (CLR) este activă pe zero logic;
- intrarea serială de date pentru deplasarea spre stânga este LSI ;
- intrarea serială de date pentru deplasarea spre dreapta este RSI ;
- operațiile de deplasare la stânga, deplasare la dreapta și încărcare paralelă sunt efectuate sincron cu semnalul de ceas CLK , pe tranziția pozitivă;
- pentru comanda de memorare, semnalul de ceas nu are efect asupra funcționării circuitului;



2.4. Aplicații ale registrelor

• **Registru în inel.** Am arătat în fig. 3 că încărcarea serie a unui registru de deplasare se face cu pierderea informației inițiale din registru. Dacă se conectează ieșirea serială la intrarea serie se obține un registru în inel care recirculează la nesfârșit o secvență binară. Secvența dorită poate fi încărcată paralel sau serie, înainte de închiderea legăturii dintre ieșirea serială și intrarea serială.

Ca aplicații ale registrelor în inel se pot enumera: realizarea de numărătoare Johnson; realizarea unor divizoare digitale de frecvență; realizarea unor lumini dinamice.

Un exemplu de lumină dinamică este prezentată în figura 5. Funcționarea acestei scheme este relativ simplă:

- Imediat după conectarea sursei de alimentare, circuitul format din $C2$, $R1$ și $P2$, generează o comandă de încărcare paralelă a stărilor logice fixate de către $K1 \div K4$.
- Apariția comenzii de încărcare paralelă se explică astfel: inițial $C2$ este descărcat și intrarea porții $P2$ se află în zero logic, iar ieșirea în unu logic, deci se obține $CM=1$.
- La scurt timp după conectarea sursei de alimentare, $C2$ se încarcă, la intrarea porții $P2$ ajunge o tensiune suficient de mare să fie interpretată ca unu logic - fapt ce conduce la apariția unui zero logic la ieșire. Deci, $CM=0$, ceea ce înseamnă că registrul primește o comandă de încărcare serie cu deplasare la dreapta.
- Datorită conexiunii dintre Q_D (ieșire serială) și SI (intrare serială), informația încărcată paralel va fi recirculată în inel;
- Comutatoarele $K1 \div K4$ sunt folosite pentru stabilirea secvenței binare, starea lor este citită doar în primele momente de după conectarea sursei de alimentare.
- Dacă secvența binară încărcată paralel a fost 1000 pe bareta de LED-uri se va deplasa un LED aprins, iar dacă secvența a fost 0111, se va deplasa un LED stins;
- semnalul de ceas este asigurat prin intermediul unui oscilator realizat cu un inversor trigger Schmitt (poarta $P1$).

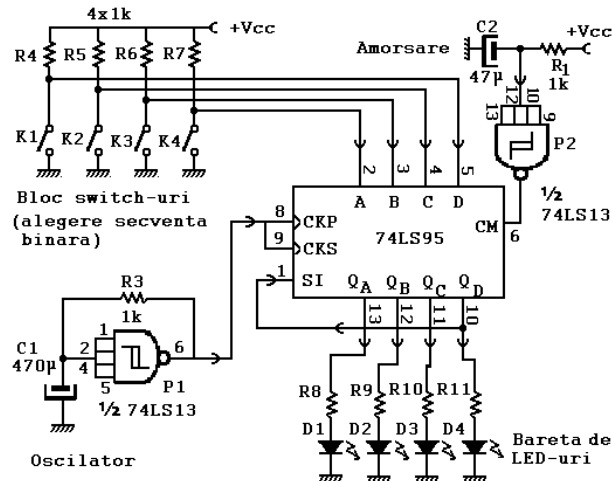


Fig. 5. Schema logică a unei lumini dinamice

• **Generarea de numere pseudoaleatoare.** Acolo unde este nevoie de generarea unor numere aleatoare, o soluție de compromis constă în generarea unor numere pseudoaleatoare, adică în generarea unei secvențe foarte lungi de numere. Deși succesiunea numerelor este mereu aceeași, datorită lungimii mari a secvenței, utilizatorul va avea senzația că numerele sunt generate aleator.

O posibilitate de generare a unei secvențe de numere pseudoaleatoare constă în utilizarea unui registru de deplasare la care se aplică o reacție liniară (Linear Feedback Shift Register). Spre deosebire de registrul în inel – unde reacția este realizată printr-o legătură între ieșirea serială de date și intrarea serială de date –, în cazul generatoarelor de numere pseudoaleatoare, reacția este asigurată printr-o poartă sau exclusiv conectată ca în figura 6.

Pentru fiecare tranziție activă a semnalului de ceas, starea registrului se modifică pe de o parte datorită faptului că informația este deplasată la dreapta iar pe de altă parte datorită faptului că intrarea serială de date primește informația $Q_n \oplus Q_m$.

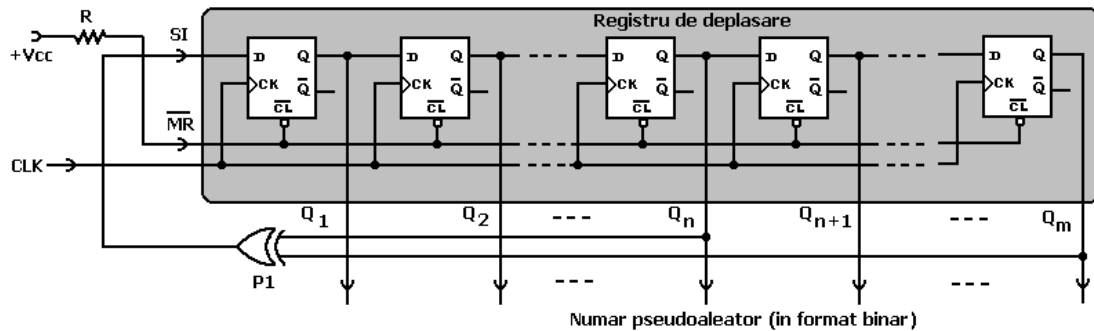


Fig. 6. Schema de principiu a unui generator de numere pseudoaleatoare

Din analiza tabelului 1 (unde se prezintă legătura dintre poziția reacției (n), lungimea registrului (m) și numărul de stări distincte), se observă că pentru dimensiuni mari ale registrului se obțin secvențe lungi și astfel se poate crea aparența de generare aleatoare a numerelor.

Tabelul 1 .

Lungime registru (m)	Poziție reacție (n)	Numărul de stări distincte	Lungime registru (m)	Poziție reacție (n)	Numărul de stări distincte
4	3	16	20	17	1.048.475
5	3	31	21	19	2.097.151
6	5	63	22	21	4.194.303
7	6	127	23	18	8.388.607
9	5	511	25	22	33.554.431
10	7	1.023	28	25	268.435.455
11	9	2.047	29	27	536.870.911
15	14	32.767	31	28	2.147.483.647

• **Conversia între formatele de reprezentare a informației digitale.** Informația vehiculată în sistemele digitale se face fie în format serial fie în format paralel. Trecerea de la un format la altul se face cu ajutorul registrelor de deplasare. Modul concret de utilizare se va studia într-o altă lucrare de laborator.

2.5. Descrierea registrelor cu ajutorul limbajului VHDL

♦ **Exemplul 1:** Descrierea în limbaj VHDL a unui registru pe 8 biți cu încărcare paralelă pe frontul pozitiv al semnalului de ceas. Intrarea de ștergere a registrului este activă pe nivelul de unu logic.

Observații	Codul VHDL
Secțiune dedicată includerii de librării.	-- Registru pe 8 biți cu încărcare paralelă library IEEE; use IEEE.std_logic_1164.all;
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: reg_8 ; - intrarea de ceas: clk ; - comandă încărcare paralelă: ld ; - intrări paralele de date: din ; - ieșiri paralele de date: dout ;	entity reg8 is port (clk: in std_logic; reset: in std_logic; ld: in std_logic; din: in std_logic_vector(7 downto 0); dout: out std_logic_vector(7 downto 0)); end reg8;
Secțiune dedicată descrierii arhitecturii. În cazul de față: - numele arhitecturii este: arh8 ; - arhitectura este asociată cu entitatea: reg8 ; Observații: - procesul p1 este sensibil la intrarea de ceas clk ; - resetul este activ pe unu logic;	architecture arh8 of reg8 is begin p1: process (clk) begin if (reset = '1') then dout <= "00000000"; elsif (clk'event and clk='1') then if (ld = '1') then dout <= din; end if; end if; end process p1; end arh8;

♦ **Exemplul 2:** Descrierea în limbaj VHDL a unui registru cu intrare serie și ieșire paralelă pe 8 biți. Intrarea de ceas este activă pe tranziția pozitivă. Circuitul nu are intrare de ștergere.

Observații	Codul VHDL
Secțiune dedicată includerii de librării	-- Exemplu de registru de deplasare library IEEE; use IEEE.std_logic_1164.all;
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: reg_8 ; - intrarea de ceas: clk ; - intrarea seriala de date: si ; - ieșirile de date: dq ;	entity reg_8 is port (si, clk : in std_logic; dq : out std_logic_vector(7 downto 0)); end reg_8;
Secțiune dedicată descrierii arhitecturii. În cazul de față: - numele arhitecturii este: arh_reg ; - arhitectura este asociată cu entitatea: reg_8 ; - descrierea funcționării se face cu un proces ce este sensibil la semnalul de ceas clk ; Observații: Deplasarea se face pe frontul crescător al semnalului de ceas; Data de pe intrarea serie este copiată în prima celulă a registrului de memorie.	architecture arh_reg of reg_8 is begin process (clk) begin if (clk ' event) and (clk='1') then dq(7 downto 1) <= dq(6 downto 0); dq(0) <= si ; -- descriere alternativa -- dq <= dq(6 downto 0) & si; end if ; end process ; end arh_reg ;

♦ **Exemplul 3:** Descrierea în limbaj VHDL a unui registru pe 4 biți cu încărcare paralelă și posibilitatea de deplasare a informației spre stânga sau spre dreapta. Metoda folosită în acest exemplu este preferabilă celei anterioare.

Observații	Codul VHDL
Package -ul conține elemente ce trebuie recunoscute în mai multe proiecte. Elementele dintr-un pachet sunt recunoscute într-un proiect dacă se face apel la instrucțiunea use . În cazul de față se definește un pachet pentru a specifica formatul bit4 .	library IEEE; use IEEE.std_logic_1164.all; package reg_types is subtype bit4 is std_logic_vector(3 downto 0); end reg_types;
Secțiune dedicată includerii de librării. Se remarcă faptul că se include și pachetul descris mai sus.	-- Exemplu de registru pe 4 biți library IEEE; use IEEE.std_logic_1164.all; use WORK.reg_types.all;
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: reg_A ; - intrarea de ceas: clk ; - comandă încărcare paralelă: load ; - intrare pentru specificarea sensului de deplasare a informației din registru: shift_sens ; - intrări paralele de date: din ; - ieșiri paralele de date: dout ;	entity reg_A is port (clk, shift_sens, load : in std_logic; din : in bit4; dout : inout bit4); end nr_A;
Secțiune dedicată descrierii arhitecturii. În cazul de față: - numele arhitecturii este: arh_A ; - arhitectura este asociată cu entitatea: reg_A ; Observații: - procesul pr_1 este responsabil de menținerea stării registrului; - procesul: pr_2 este folosit pentru transferul stării registrului spre ieșirile circuitului; - pentru descrierea deplasării spre dreapta, în locul declarațiilor: temp_val(2 downto 0) <= temp_val(3 downto 1); temp_val(3) = '0'; se poate folosi temp_val <= '0' & temp_val(3 downto 1); - pentru descrierea deplasării spre stânga, în locul declarațiilor: temp_val(3 downto 1) <= temp_val(2 downto 0); temp_val(0) = '0'; se poate folosi temp_val <= temp_val(2 downto 0) & '0';	architecture arh_A of reg_A is signal temp_val: bit4; begin -- procese ptr. descriere functionare registru pr_1: process (shift_sens, load, din, dout) begin if load = '1' then temp_val <= din ; elsif shift_sens = '1' then -- deplasare spre dreapta temp_val(2 downto 0) <= temp_val(3 downto 1); temp_val(3) = '0' ; else -- deplasare spre stanga temp_val(3 downto 1) <= temp_val(2 downto 0); temp_val(0) <= '0' ; end if; end process pr_1 ; pr_2: process begin wait until clk 'event and clk ='1' dout <= temp_val ; end process pr_2 ; end arh_A ;



3. Desfășurarea lucrării

3.1. Studiul registrelor realizate în structuri integrate.

- A. Cum trebuie conectat un circuit 74LS95 pentru a obține un registru de deplasare bidirecțional, cu intrare serie și ieșire paralelă ?
- B. Referitor la schema din figura 5, răspundeți la următoarele întrebări:
 - Pentru a obține efectul de "lumină curgătoare" este nevoie de mai multe LED-uri așezate în linie (bareta luminoasă). Dacă menținem capacitatea registrului la 4 biți cum trebuie conectate din punct de vedere electric LED-urile pentru a obține o bareta luminoasă de 32 poziții ?
 - Care este informația ce trebuie încărcată paralel dacă dorim deplasarea unui singur LED stins ? Dar pentru un singur LED aprins ?
 - Asupra cărei componente trebuie acționat pentru a reduce viteza de curgere ?
 - Dacă schema este pornită și deplasează un LED aprins, ce trebuie făcut ca ea să deplaseze un LED stins ?
 - Care va fi efectul vizibil pe bareta de LED-uri dacă legătura dintre Qd și SI se face printr-un inversor logic ?
- C. Folosind registre de tip 74LS95 și alte circuite auxiliare necesare, concepeți o schemă de lumină dinamică cu 8 LED-uri știind că LED-ul aprins trebuie să parcurgă următorul ciclu: LED0, LED1, LED2, LED3, LED4, LED5, LED6, LED7, LED6, LED5, LED4, LED3, LED2, LED1, LED0, LED1 ..., cu alte cuvinte sensul de deplasare se schimbă după ce LED-ul aprins a ajuns la una din extremitățile baretei.

3.2. Utilizarea ISE WebPack pentru descrierea aplicațiilor sub formă de scheme logice

- A. Folosind facilitatea ISE-WebPack prin care se permite descrierea proiectelor la nivel de schemă logică se cere implementarea și testarea schemelor din figurile 1, 2 și 4 pe macheta de laborator cu CPLD;
- B. Folosind facilitatea ISE-WebPack prin care se permite descrierea proiectelor la nivel de schemă logică se cere implementarea și testarea unui generator de numere pseudoaleatoare (vezi figura 6) cu $m=4$ și $n=2$.

3.3. Utilizarea limbajului VHDL

- A. Implementați și verificați pe macheta de laborator cu CPLD, descrierile în limbaj VHDL prezentate ca exemple. Cum se execută funcția de reset la registrul din exemplul 1 ?
- B. Modificați descrierea VHDL de la exemplul 2 pentru adăugarea unei intrări de ștergere cu execuție asincronă față de semnalul de ceas.
- C. În exemplul 3, deplasarea informației ce a fost încărcată paralel se face cu pierderea treptată (câte un bit la fiecare perioadă a semnalului de ceas) a datelor. Modificați descrierea VHDL de la exemplul 3 astfel încât informația încărcată paralel să fie rotită (recirculată) la nesfârșit.
- D. Concepeți o descriere comportamentală pentru circuitul 74LS194 după care verificați corectitudinea ei pe macheta de laborator cu CPLD.
- E. Concepeți o descriere în limbaj VHDL și implementați pe macheta de laborator cu CPLD, o lumină dinamică pe 8 biți care să funcționeze similar cu cea din figura 5.
- F. Concepeți o descriere în limbaj VHDL și implementați pe macheta de laborator cu CPLD, o lumină dinamică cu 8 LED-uri știind că LED-ul aprins trebuie să parcurgă următorul ciclu: ... , LED0, LED1, LED2, LED3, LED4, LED5, LED6, LED7, LED6, LED5, LED4, LED3, LED2, LED1, LED0, LED1 ..., cu alte cuvinte LED-ul aprins se deplasează de la stânga la dreapta și când atinge capătul baretei se deplasează în sens invers.
- G. Concepeți o descriere în limbaj VHDL și implementați pe macheta de laborator cu CPLD, o lumină dinamică cu 8 LED-uri pentru a obține următorul efect: bareta se aprinde de la stânga la dreapta - dând efectul de umplere, iar după ce întreaga bareta este aprinsă se stinge de la dreapta la stânga - dând efectul de golire.
- H. Concepeți o descriere în limbaj VHDL și implementați pe macheta de laborator cu CPLD, un generator de numere pseudoaleatoare cu o secvență de cel puțin 63 numere. Afișarea numerelor se va face în cod binar pe bareta de LED-uri sau în zecimal pe două afișaje cu 7 segmente.

4. Indicații privind modul de lucru

Pentru fiecare aplicație este necesară deschiderea unui nou proiect după metodologia prezentată într-o lucrare de început.

Toate aplicațiile din această lucrare necesită doar un singur fișier sursă (fie schemă logică, fie sursă VHDL) și un singur fișier de constrângeri .

Referitor la conectarea intrărilor și a ieșirilor din circuit facem următoarele precizări:

- pentru comanda intrărilor de date se vor folosi switch-urile SW1, SW2, SW3, SW4;
- pentru afișarea informației din registru se vor folosi LED-urile LED1, LED2, LED3 și LED4;
- pentru intrarea de ceas se folosește BTN1, pentru intrarea de reset BTN2 iar pentru intrarea de încărcare paralelă BTN3;
- pentru intrarea de comandă a sensului de deplasare se va utiliza SW8;
- pentru intrarea serială de date se va utiliza BTN8;

Toate aceste precizări trebuie să se regăsească în conținutul fișierului de constrângeri, alcătuirea sa, sperăm, nu mai ridică nici o problemă.

Referitor la obținerea semnalului de comandă a intrărilor de ceas, facem următoarele precizări:

- Petru a crea un efect perceptibil de lumină dinamică este nevoie ca frecvența semnalului ce comandă deplasarea informației din registru să fie sub 2Hz. Dacă frecvența este prea mare, vom percepe toată bareta aprinsă.
- Pentru comanda intrării de ceas se folosește oscilatorul de pe macheta de laborator cu CPLD. Deoarece frecvența acestuia este de 25,1758MHz, el trebuie mai întâi divizat în frecvență și abia apoi folosit pentru comanda intrării de ceas a registrului de deplasare. Dacă ne propunem un semnal cu frecvența de 1Hz, constanta de divizare este egală cu 25.175.000.
- Din punct de vedere al sintezei este mai bine să folosim o constantă de divizare ce reprezintă o putere a lui 2, spre exemplu $2^{23} = 8.388.608$, ceea ce înseamnă o frecvență de ieșire de cca. 3 Hz, valoare acceptabilă pentru aplicațiile propuse.
- În fișierul VHDL, pe lângă descrierea propriu-zisă a funcționării, se va introduce un proces pentru divizarea semnalului preluat de la oscilatorul machetei.
- Modul de intercalare al acestui process cu restul aplicației se prezintă pe coloana alăturată. Cea mai mare divizare în frecvență se obține dacă semnalul de atac al registrului **clk_reg** este preluat de la cel mai semnificativ bit al semnalului **cnt**. Pentru aceasta este necesar să facem o declarație de forma:


```
clk_reg <= cnt(22);
```
- Starea registrului de deplasare se afișează pe bareta de LED-uri de pe macheta de laborator cu CPLD.

```
-- Exemplu de structura
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

entity lumina_dinamica is
port (
    osc_25M : in std_logic;
    -- se declara restul de intrari si
    -- iesiri necesare
);
end lumina_dinamica;

architecture arh_2 of lumina_dinamica is
    signal cnt: std_logic_vector(22 downto 0);
    signal clk_reg: std_logic;
    -- alte declaratii,daca este cazul
begin
    -- proces pentru divizare ceas
    divizare: process (osc_25M)
    begin
        if (osc_25M 'event and osc_25M='1') then
            cnt <= cnt + 1;
        end if;
    end process divizare;
    clk_reg <= cnt(22);

    -- alte procese sau declaratii concurente
    -- pentru descrierea aplicatiilor cerute
    -- in lucrare

end arh_2;
```

